

PREMIER UNIVERSITY, CHATTOGRAM

Department of Computer Science & Engineering



Project Report

Course Name : Big Data Analytics laboratory

Course Code : CSE 4346

Name of Report : Big Data Analysis of Amazon Product Dataset
Using Hadoop, Apache Pig, and MapReduce

Date of Submission : 15/09/2026

Submitted By:	
Name	: Sushen Chakraborty
ID	: 0222220005101157
Semester	: 8 th
Batch	: 42 nd
Session	: Spring 2026
Section : C	

Submitted To : Md. Tamim Hossain Lecturer Department of Computer Science and Engineering Premier University, Chattogram.	Remarks
---	----------------

Table of Contents

Table of Contents	2
1. Introduction	3
2. Problem Statement	4
3. Dataset Description	5
4. Hadoop Environment Setup	7
5. Apache Pig Analysis	9
5.1 Rating Analysis	9
5.2 Review Analysis	11
5.3 Category Analysis	13
5.4 Price Analysis	15
5.5 Purchase Analysis	16
6. MapReduce Analysis	19
6.1 Review Range Analysis	19
6.2 Top Price Analysis	23
6.3 Purchase Ranking Analysis	28
6.4 Product Popularity Analysis	32
7. Results and Discussion	36
8. Conclusion	38

1. Introduction

The rapid growth of e-commerce platforms has generated a massive amount of product-related data from millions of users worldwide. Online marketplaces continuously collect information related to products, customer reviews, ratings, prices, categories, and purchasing activities. This large-scale data contains valuable information that can help understand customer behavior, product performance, and market trends.

However, processing and analyzing such large datasets manually is challenging due to the increasing volume, variety, and complexity of information. Traditional data processing methods often face limitations when handling large amounts of structured and semi-structured data. Therefore, big data technologies are required to store, process, and analyze large datasets efficiently.

Big data technologies provide effective solutions for distributed storage and parallel data processing. Apache Hadoop is one of the widely used frameworks for big data processing. Hadoop provides a distributed computing environment through components such as Hadoop Distributed File System (HDFS) and MapReduce. HDFS allows large datasets to be stored across multiple nodes, while MapReduce enables efficient processing of data through parallel computation.

This project focuses on analyzing an Amazon product dataset using the Hadoop ecosystem. The dataset contains approximately 1.4 million product records with important attributes such as product identifiers, product titles, ratings, customer reviews, prices, categories, and monthly purchase information.

Apache Pig and MapReduce are used in this project to perform different analytical operations on Amazon product information. Apache Pig provides a high-level data processing platform that simplifies data analysis tasks using Pig Latin scripts. MapReduce is used to develop customized Java-based programs for processing and analyzing large-scale product data.

The main purpose of this project is to extract meaningful insights from Amazon product data by analyzing important attributes such as product ratings, customer reviews, product categories, prices, and purchase activities. The analysis helps to identify product patterns, customer engagement levels, price variations, and popularity trends within the dataset.

Through this project, the practical application of Hadoop-based big data analytics is demonstrated. The implementation shows how distributed processing technologies can be used to handle large datasets and generate useful information for data-driven decision-making.

2. Problem Statement

The rapid expansion of e-commerce platforms has resulted in the generation of a large amount of product-related data. This data includes product information, customer reviews, ratings, pricing details, product categories, and purchasing activities. Extracting useful information from such a large dataset requires efficient storage and processing techniques.

The objective of this project is to analyze a large Amazon product dataset using Hadoop-based big data technologies. The project focuses on applying distributed data processing techniques to handle approximately 1.4 million product records and extract meaningful patterns from the available information.

The project aims to achieve the following objectives:

- To store and manage large-scale Amazon product data using Hadoop Distributed File System (HDFS).
- To perform high-level data analysis operations using Apache Pig for extracting information related to product ratings, reviews, categories, prices, and purchase activities.
- To develop customized analytical solutions using Java-based Hadoop MapReduce programs for processing specific product-related problems.
- To analyze customer engagement patterns based on product reviews and ratings.
- To identify price-related patterns and discover products with higher price values from the dataset.
- To analyze purchasing activity and identify products with higher customer purchase interaction.
- To classify products based on review ranges and evaluate product popularity using customer review information.
- To understand how Hadoop ecosystem technologies can efficiently process and analyze large-scale datasets.

The project demonstrates the practical application of Hadoop-based analytics for handling large volumes of e-commerce data. By using HDFS, Apache Pig, and MapReduce, the project aims to transform raw product data into meaningful analytical results that can support data-driven decision-making.

3. Dataset Description

The dataset used in this project is an Amazon product dataset containing information about products, customer ratings, reviews, prices, categories, and purchasing activities.

Dataset Name:

`amazon_products.csv`

Dataset Location in HDFS:

/amazon/input/amazon_products.csv

The dataset contains the following attributes:

Attribute	Description
asin	Unique product identifier
title	Product title
imgUrl	Product image URL
productURL	Product page URL
stars	Product rating
reviews	Number of customer reviews
price	Product price
listPrice	Original listed price
category_id	Product category identifier
isBestSeller	Best seller information
boughtInLastMonth	Number of purchases in the last month

Dataset Size

The dataset used in this project contains approximately 1.4 million product records collected from Amazon product data.

The large volume of data makes it suitable for big data processing using Hadoop technologies. The dataset was stored in HDFS and processed using Apache Pig and Java-based MapReduce programs.

Total Records:

Dataset	Number of Records
Amazon Product Dataset	Approximately 1.4 Million Records

```
H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>hdfs dfs -cat /amazon/input/amazon_products.csv > amazon_products_local.csv

H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>find /c /v "" amazon_products_local.csv

----- AMAZON_PRODUCTS_LOCAL.CSV: 1426338

H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>
```

Figure 3.1: Dataset Record Count Output

4. Hadoop Environment Setup

The project was implemented using the following software environment:

Software	Version
Apache Hadoop	3.2.4
Apache Pig	0.18.0
Java	1.8.0_411

HDFS Input Location

/amazon/input/amazon_products.csv

Sl. No.	HDFS Output Directory	Purpose
1	/amazon/output/rating_analysis	Stores output of product rating analysis
2	/amazon/output/review_analysis	Stores output of product review analysis
3	/amazon/output/category_analysis	Stores output of category-wise product analysis
4	/amazon/output/price_analysis	Stores output of product price analysis
5	/amazon/output/purchase_analysis	Stores output of monthly purchase analysis
6	/amazon/output/review_range_analysis	Stores output of review range classification using MapReduce
7	/amazon/output/top_price_analysis	Stores output of top expensive product analysis using MapReduce
8	/amazon/output/purchase_ranking_analysis	Stores output of product purchase ranking analysis
9	/amazon/output/product_popularity_analysis	Stores output of product popularity analysis based on reviews

Hadoop Version

```
H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>hadoop version
Hadoop 3.2.4
Source code repository Unknown -r 7e5d9983b388e372fe640f21f048f2f2ae6e9eba
Compiled by ubuntu on 2022-07-12T11:58Z
Compiled with protoc 2.5.0
From source with checksum ee031c16fe785bbb35252c749418712
This command was run using /C:/hadoop/share/hadoop/common/hadoop-common-3.2.4.jar
```

Figure 4.1: Hadoop Version Output

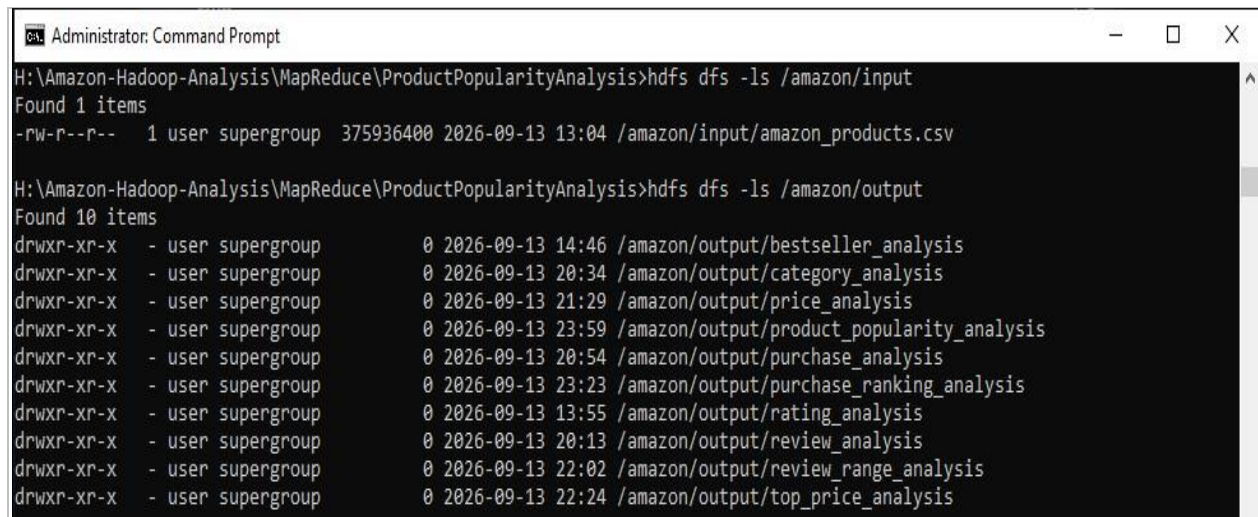
Pig and Java Version

```
H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>pig -version
Apache Pig version 0.18.0 (r1926484)
compiled Jun 16 2025, 10:38:21

H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>java -version
java version "1.8.0_411"
Java(TM) SE Runtime Environment (build 1.8.0_411-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.411-b09, mixed mode)
```

Figure 4.2: Pig and Java Version Output

HDFS Input Architecture



The screenshot shows a Windows Command Prompt window titled "Administrator: Command Prompt". The user is in the directory H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis. They run the command `hdfs dfs -ls /amazon/input` and receive the output: "Found 1 items" followed by a single file entry: `-rw-r--r-- 1 user supergroup 375936400 2026-09-13 13:04 /amazon/input/amazon_products.csv`. Then they run `hdfs dfs -ls /amazon/output` and receive the output: "Found 10 items" followed by a list of 10 files with their permissions, owner, group, size, modification time, and path.

```
Administrator: Command Prompt
H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>hdfs dfs -ls /amazon/input
Found 1 items
-rw-r--r-- 1 user supergroup 375936400 2026-09-13 13:04 /amazon/input/amazon_products.csv

H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>hdfs dfs -ls /amazon/output
Found 10 items
drwxr-xr-x - user supergroup 0 2026-09-13 14:46 /amazon/output/bestseller_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 20:34 /amazon/output/category_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 21:29 /amazon/output/price_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 23:59 /amazon/output/product_popularity_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 20:54 /amazon/output/purchase_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 23:23 /amazon/output/purchase_ranking_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 13:55 /amazon/output/rating_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 20:13 /amazon/output/review_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 22:02 /amazon/output/review_range_analysis
drwxr-xr-x - user supergroup 0 2026-09-13 22:24 /amazon/output/top_price_analysis
```

Figure 4.3: HDFS Input Directory Structure

```
H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>tree /F
Folder PATH listing for volume New Volume
Volume serial number is A454-C588
H:
    PopularityDriver.class
    PopularityDriver.java
    PopularityMapper.class
    PopularityMapper.java
    PopularityReducer.class
    PopularityReducer.java
    ProductPopularityAnalysis.jar

No subfolders exist
```

Figure 4.4: HDFS Input File Details

5. Apache Pig Analysis

Apache Pig is a high-level data processing platform used for analyzing large datasets in Hadoop. Pig Latin scripts were used in this project to perform data loading, filtering, grouping, and aggregation operations on the Amazon product dataset.

Apache Pig reduces the complexity of writing complex MapReduce programs by providing a simple scripting language. In this project, Apache Pig was used to analyze different product attributes such as ratings, reviews, categories, prices, and purchase activities.

The following analyses were performed using Apache Pig:

1. Rating Analysis
2. Review Analysis
3. Category Analysis
4. Price Analysis
5. Purchase Analysis

5.1 Rating Analysis

Objective

The objective of Rating Analysis is to analyze the distribution of products based on their star ratings. This analysis helps to understand how products are distributed according to customer ratings.

Process

The following steps were performed:

1. Load the Amazon product dataset from HDFS.
2. Extract the rating (stars) attribute.
3. Filter valid rating values.
4. Group products based on rating.
5. Count the number of products for each rating value.
6. Store the final output in HDFS.

Pig Script Logic

Input:

/amazon/input/amazon_products.csv

Output:

/amazon/output/rating_analysis

Output Format

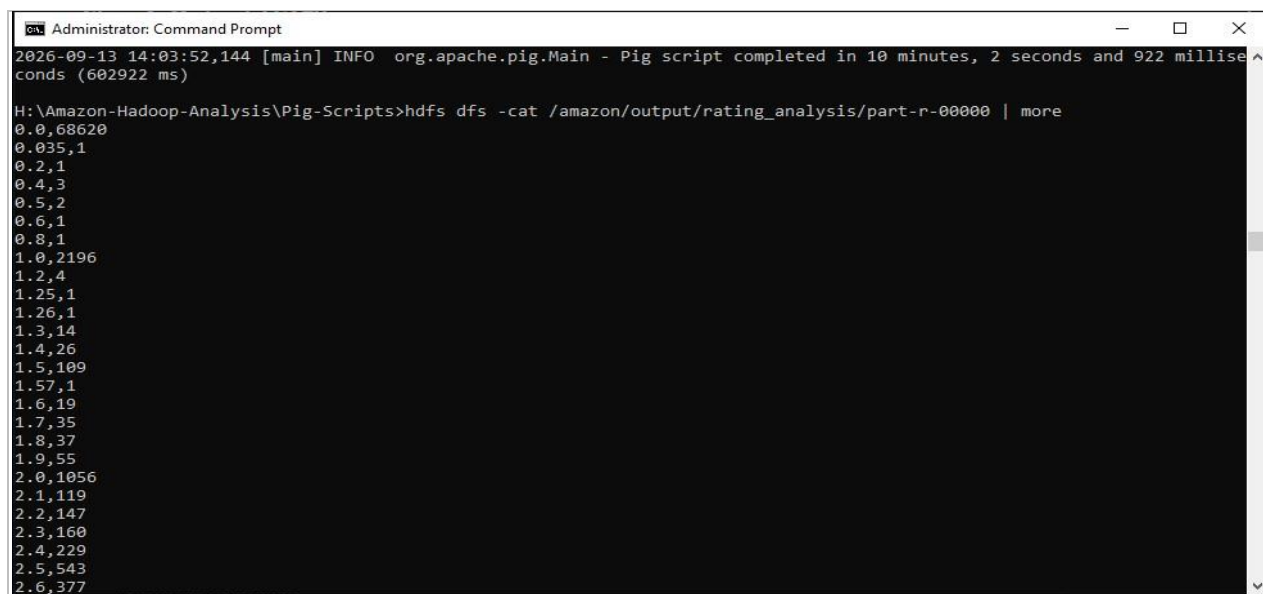
Rating	Number of Products
1.0	Count
2.0	Count
3.0	Count
4.0	Count
5.0	Count

Rating Analysis Pig Script

```
products = LOAD '/amazon/input/amazon_products.csv'
USING PigStorage(',')
AS (
  asin:chararray,
  title:chararray,
  imgUrl:chararray,
  productURL:chararray,
  stars:double,
  reviews:int,
  price:chararray,
  listPrice:chararray,
  category_id:int,
  isBestSeller:chararray,
  boughtInLastMonth:int
);
filtered = FILTER products BY stars IS NOT NULL AND stars >= 0 AND stars <= 5;
grouped = GROUP filtered BY stars;
result = FOREACH grouped GENERATE
  group AS rating,
  COUNT(filtered) AS total_products;
STORE result INTO '/amazon/output/rating_analysis' USING PigStorage(',');
```

Figure 5.1: Rating Analysis Pig Script

Rating Analysis Output



```
Administrator: Command Prompt
2026-09-13 14:03:52,144 [main] INFO org.apache.pig.Main - Pig script completed in 10 minutes, 2 seconds and 922 millise
conds (602922 ms)

H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -cat /amazon/output/rating_analysis/part-r-00000 | more
0.0,68620
0.035,1
0.2,1
0.4,3
0.5,2
0.6,1
0.8,1
1.0,2196
1.2,4
1.25,1
1.26,1
1.3,14
1.4,26
1.5,109
1.57,1
1.6,19
1.7,35
1.8,37
1.9,55
2.0,1056
2.1,119
2.2,147
2.3,160
2.4,229
2.5,543
2.6,377
```

Figure 5.2: Rating Analysis Output

5.2 Review Analysis

Objective

The objective of Review Analysis is to analyze products based on the number of customer reviews available in the Amazon product dataset. Customer reviews are an important indicator of product interaction and customer engagement. Products with a higher number of reviews generally receive more attention from customers compared with products having fewer reviews.

This analysis helps to understand the level of customer participation for different products. By analyzing review information, it becomes possible to identify product engagement patterns and evaluate how customers interact with products on the e-commerce platform.

Process

The following steps were performed to complete the Review Analysis:

1. Load the Amazon product dataset from HDFS.
2. Extract the required product attributes, including product ID (asin) and customer review count (reviews).
3. Filter the dataset to remove invalid or missing review values to ensure accurate analysis.
4. Process the extracted review information using Apache Pig operations.
5. Generate product-wise review details from the processed dataset.
6. Store the final analysis output in the Hadoop Distributed File System (HDFS) for further evaluation.

Pig Script Logic

The Amazon product dataset was loaded into Apache Pig using the HDFS input path. The required fields were selected from the dataset, and products containing valid review information were processed.

The analysis focused on extracting the relationship between individual products and their corresponding customer review counts. The generated output provides product-level information that can be used to understand customer engagement patterns.

Input:

/amazon/input/amazon_products.csv

Output:

/amazon/output/review_analysis

Output Description

The output contains two main attributes:

Product ID (asin):

A unique identifier assigned to each product in the Amazon dataset.

Review Count:

The total number of customer reviews received by each product.

The generated output provides a clear view of customer review distribution across different products. This information can be used for further analysis of product popularity and customer interaction.

Result Discussion

The Review Analysis successfully extracted product-wise review information from the Amazon dataset. The output demonstrates how Apache Pig can efficiently process large-scale product data and generate meaningful information from raw datasets.

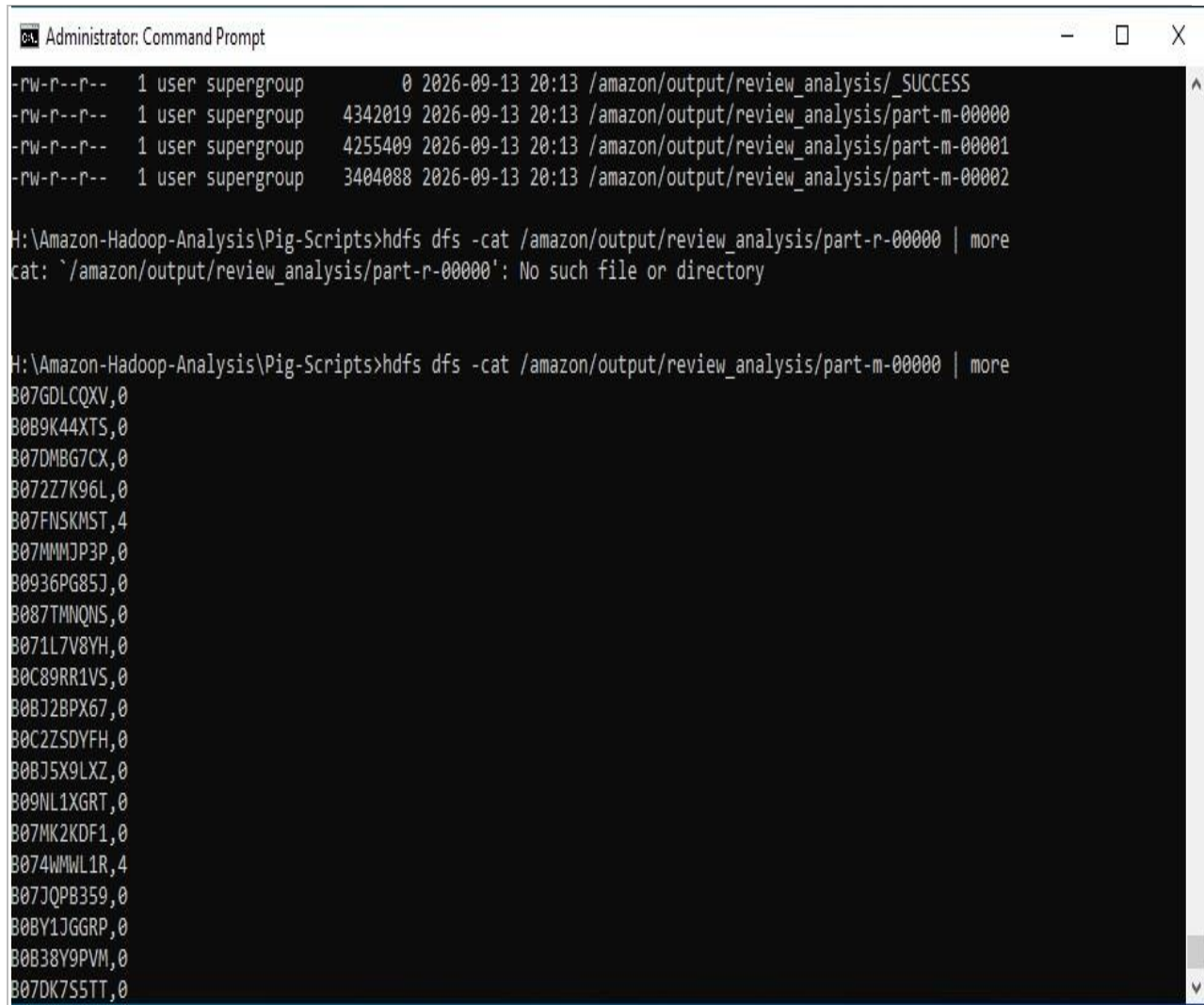
The analysis provides a foundation for understanding customer engagement patterns. Products with higher review counts can be identified and further analyzed for popularity, customer interest, and market performance.

Review Analysis Pig Script

```
File Edit Format View Help
products = LOAD '/amazon/input/amazon_products.csv'
USING PigStorage(',')
AS (
asin:chararray,
title:chararray,
imgUrl:chararray,
productURL:chararray,
stars:double,
reviews:int,
price:chararray,
listPrice:chararray,
category_id:int,
isBestSeller:chararray,
boughtInLastMonth:int
);
filtered = FILTER products BY reviews IS NOT NULL;
result = FOREACH filtered GENERATE
asin,
reviews;
STORE result INTO '/amazon/output/review_analysis'
USING PigStorage(',');
```

Figure 5.3: Review Analysis Pig Script

Review Analysis Output



```
Administrator: Command Prompt

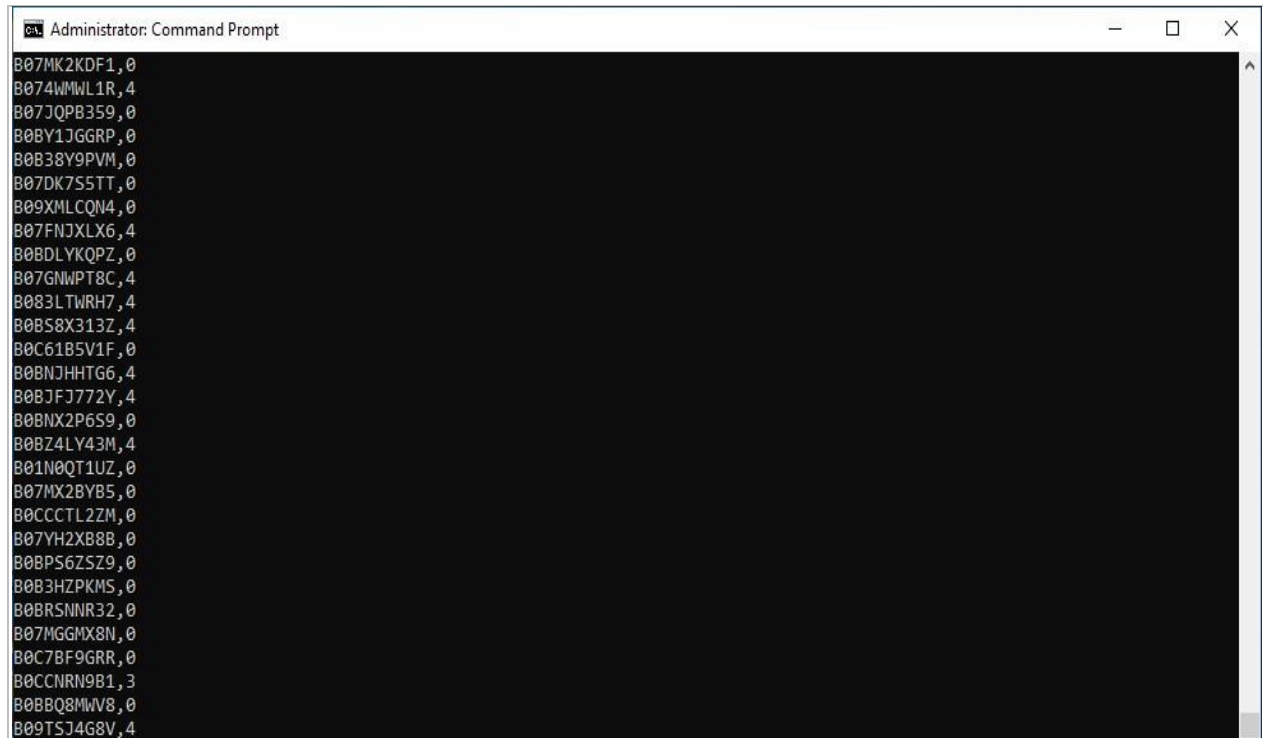
-rw-r--r--  1 user supergroup      0 2026-09-13 20:13 /amazon/output/review_analysis/ SUCCESS
-rw-r--r--  1 user supergroup 4342019 2026-09-13 20:13 /amazon/output/review_analysis/part-m-00000
-rw-r--r--  1 user supergroup 4255409 2026-09-13 20:13 /amazon/output/review_analysis/part-m-00001
-rw-r--r--  1 user supergroup 3404088 2026-09-13 20:13 /amazon/output/review_analysis/part-m-00002

H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -cat /amazon/output/review_analysis/part-r-00000 | more
cat: `/amazon/output/review_analysis/part-r-00000': No such file or directory

H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -cat /amazon/output/review_analysis/part-m-00000 | more
B07GDLQXV,0
B0B9K44XTS,0
B07DMBG7CX,0
B072Z7K96L,0
B07FNSKMST,4
B07MMMJP3P,0
B0936PG85J,0
B087TMNQNS,0
B071L7V8YH,0
B0C89RR1VS,0
B0BJ2BPX67,0
B0C2ZSDYFH,0
B0BJ5X9LXZ,0
B09NL1XGRT,0
B07MK2KDF1,0
B074WMWL1R,4
B07JQP8359,0
B0BY1JGGRP,0
B0B38Y9PVM,0
B07DK7S5TT,0
```

Figure 5.4: Review Analysis Output

Review Analysis Output (Continued)



```
Administrator: Command Prompt
B07MK2KDF1,0
B074WML1R,4
B07JQPB359,0
B0BY1JGGRP,0
B0B38Y9PVM,0
B07DK7S5TT,0
B09XMLCQN4,0
B07FNJXLX6,4
B0BDLYKQPZ,0
B07GNWPT8C,4
B083LTWRH7,4
B0BS8X313Z,4
B0C61B5V1F,0
B0BNJHHTG6,4
B0BJFJ772Y,4
B0BNX2P6S9,0
B0BZ4LY43M,4
B01N0QT1UZ,0
B07MX2BYB5,0
B0CCCTL2ZM,0
B07YH2XB8B,0
B0BPS6ZS29,0
B0B3HZPKMS,0
B0BRSNNR32,0
B07MGGMX8N,0
B0C7BF9GRR,0
B0CCNRN9B1,3
B0BBQ8MNV8,0
B09TSJ4G8V,4
```

Figure 5.5: Review Analysis Output (Continued)

5.3 Category Analysis

Objective

The objective of Category Analysis is to identify the number of products available under different categories.

Process

The following steps were performed:

1. Load Amazon product dataset.
2. Filter products with valid category IDs.
3. Group products according to category_id.
4. Count products in each category.
5. Store output in HDFS.

Pig Script Logic

Input:

/amazon/input/amazon_products.csv

Output:

/amazon/output/category_analysis

Output Format

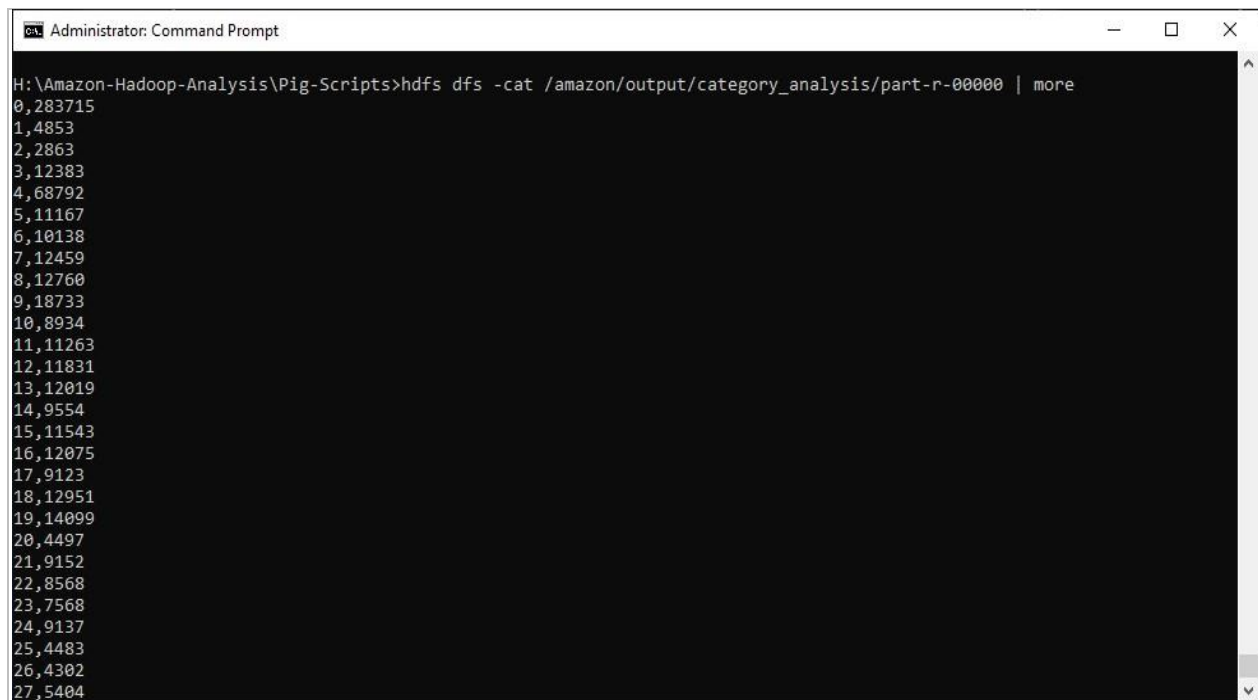
Category ID	Total Products
Category ID	Product Count

Category Analysis Pig Script

```
products = LOAD '/amazon/input/amazon_products.csv'
           USING PigStorage(',')
           AS (
             asin:chararray,
             title:chararray,
             imgUrl:chararray,
             productURL:chararray,
             stars:double,
             reviews:int,
             price:chararray,
             listPrice:chararray,
             category_id:int,
             isBestSeller:chararray,
             boughtInLastMonth:int
           );
Filtered = FILTER products BY category_id IS NOT NULL;
grouped = GROUP filtered BY category_id;
result = FOREACH grouped GENERATE
          group AS category_id,
          COUNT(filtered) AS total_products;
STORE result INTO '/amazon/output/category_analysis'
           USING PigStorage(',');
```

Figure 5.6: Category Analysis Pig Script

Category Analysis Output



```
Administrator: Command Prompt
H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -cat /amazon/output/category_analysis/part-r-00000 | more
0,283715
1,4853
2,2863
3,12383
4,68792
5,11167
6,10138
7,12459
8,12760
9,18733
10,8934
11,11263
12,11831
13,12019
14,9554
15,11543
16,12075
17,9123
18,12951
19,14099
20,4497
21,9152
22,8568
23,7568
24,9137
25,4483
26,4302
27,5404
```

Figure 5.7: Category Analysis Output

5.4 Price Analysis

Objective

The objective of Price Analysis is to analyze product distribution based on product prices in the Amazon dataset. Product price is an important factor that influences customer purchasing decisions and helps to understand the pricing patterns of products available on the platform.

This analysis helps to identify how products are distributed according to their price values. By analyzing price information, it becomes possible to understand variations in product pricing and evaluate the overall price structure within the dataset.

Process

The following steps were performed to complete the Price Analysis:

1. Load the Amazon product dataset from HDFS.
2. Extract the required product price information from the dataset.
3. Filter invalid, missing, or incomplete price values to maintain data accuracy.
4. Process product price information using Apache Pig operations.
5. Group products according to their respective price values.
6. Count the number of products associated with each price value.
7. Store the generated analysis output in Hadoop Distributed File System (HDFS).

Pig Script Logic

The Amazon product dataset was loaded into Apache Pig from the HDFS input location. The price attribute was extracted from each product record and processed to generate product distribution based on price information.

The analysis focused on identifying the relationship between product prices and the number of products available at different price levels. The generated output provides a better understanding of product pricing patterns within the Amazon dataset.

Input:

/amazon/input/amazon_products.csv

Output:/amazon/output/price_analysis

Output Format

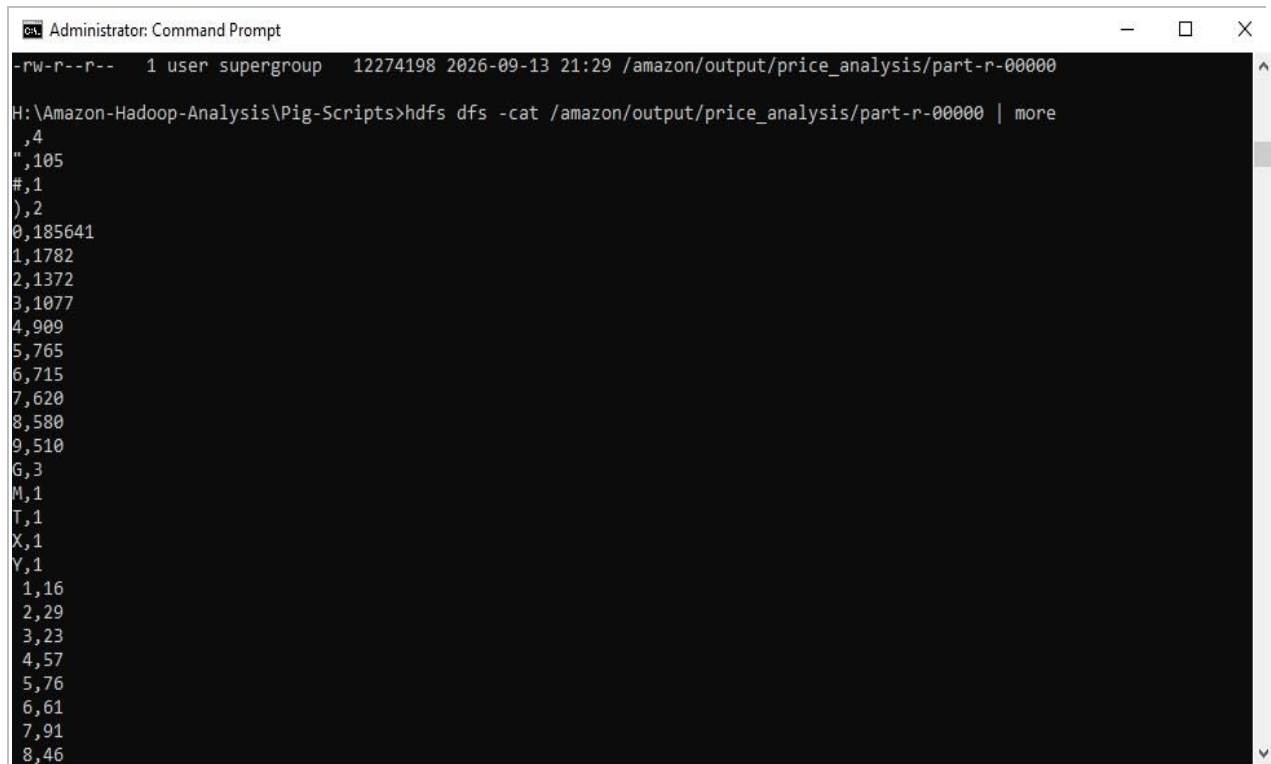
Product Price	Number of Products
Price Value	Product Count

Price Analysis Pig Script

```
products = LOAD '/amazon/input/amazon_products.csv'
USING PigStorage(',')
AS (
  asin:chararray,
  title:chararray,
  imgUrl:chararray,
  productURL:chararray,
  stars:double,
  reviews:int,
  price:chararray,
  listPrice:chararray,
  category_id:int,
  isBestSeller:chararray,
  boughtInLastMonth:int
);
filtered = FILTER products BY price IS NOT NULL;
grouped = GROUP filtered BY price;
result = FOREACH grouped GENERATE
  group AS product_price,
  COUNT(filtered) AS total_products;
STORE result INTO '/amazon/output/price_analysis'
USING PigStorage(',');
```

Figure 5.8: Price Analysis Pig Script

Price Analysis Output



```
Administrator: Command Prompt
-rw-r--r-- 1 user supergroup 12274198 2026-09-13 21:29 /amazon/output/price_analysis/part-r-00000
H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -cat /amazon/output/price_analysis/part-r-00000 | more
,4
,105
#,1
),2
0,185641
1,1782
2,1372
3,1077
4,909
5,765
6,715
7,620
8,580
9,510
G,3
M,1
T,1
X,1
Y,1
1,16
2,29
3,23
4,57
5,76
6,61
7,91
8,46
```

Figure 5.9: Price Analysis Output

5.5 Purchase Analysis

Objective

The objective of Purchase Analysis is to analyze products based on the number of purchases made in the last month. The `boughtInLastMonth` attribute provides information about recent customer purchasing activity and helps to understand product demand patterns.

This analysis helps to identify how products perform based on customer purchase interactions. By analyzing monthly purchase information, the project can evaluate product popularity and customer interest within the Amazon dataset.

Process

The following steps were performed to complete the Purchase Analysis:

1. Load the Amazon product dataset from HDFS.
2. Extract the required `boughtInLastMonth` attribute from the product records.
3. Filter valid purchase values to ensure accurate analysis.
4. Group products according to their purchase values.
5. Count the number of products available under each purchase level.
6. Generate the final purchase analysis output.
7. Store the processed output in Hadoop Distributed File System (HDFS).

Pig Script Logic

The Amazon product dataset was loaded into Apache Pig from the HDFS input location. The `boughtInLastMonth` attribute was extracted and processed to analyze product purchase activity.

The analysis focused on grouping products according to their monthly purchase values and calculating the number of products under each purchase level. This helps to identify patterns in customer purchasing behavior and product demand.

Input:

`/amazon/input/amazon_products.csv`

Output:

`/amazon/output/purchase_analysis`

Output Format

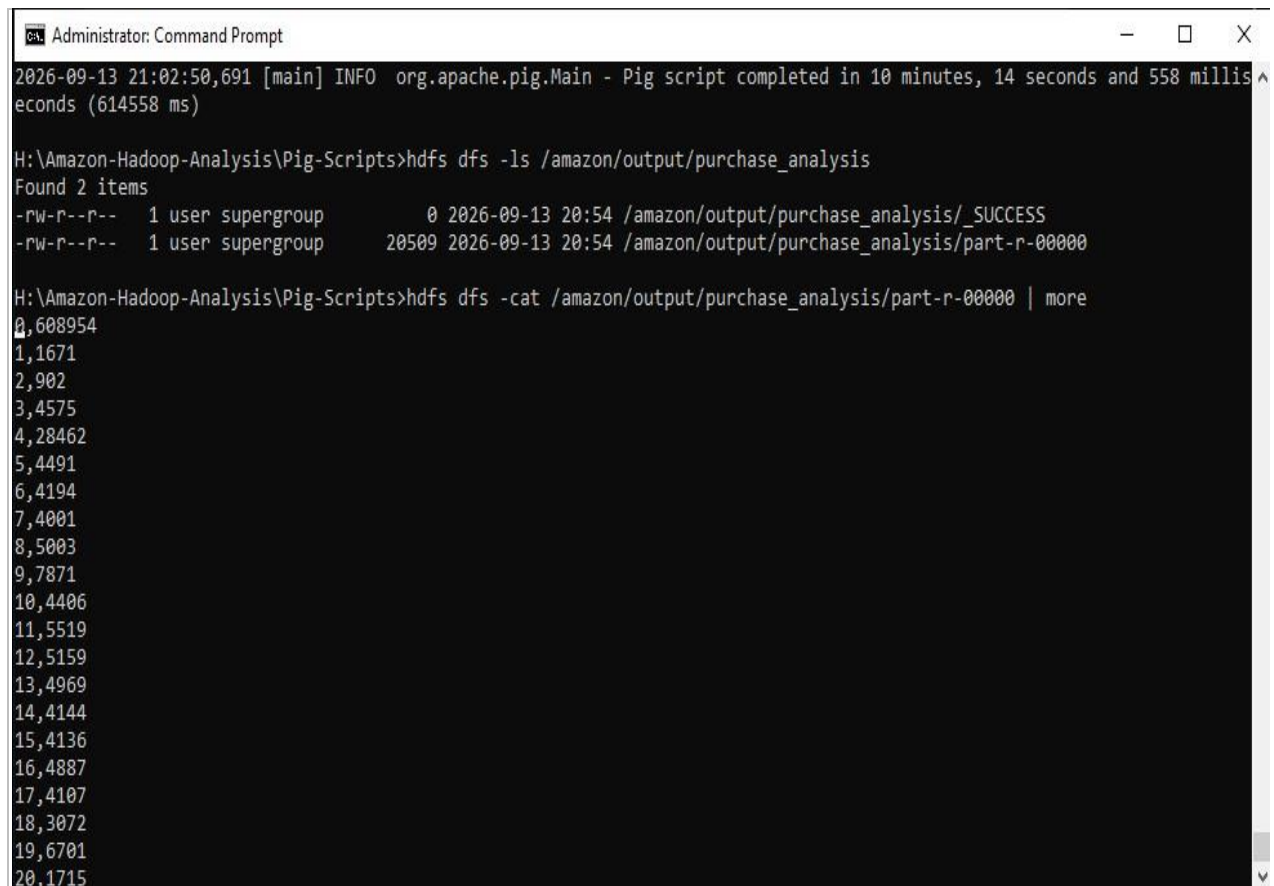
Monthly Purchase Value	Number of Products
Purchase Value	Product Count

Purchase Analysis Pig Script

```
products = LOAD '/amazon/input/amazon_products.csv'
USING PigStorage(',')
AS (
  asin:chararray,
  title:chararray,
  imgUrl:chararray,
  productURL:chararray,
  stars:double,
  reviews:int,
  price:chararray,
  listPrice:chararray,
  category_id:int,
  isBestSeller:chararray,
  boughtInLastMonth:int
);
filtered = FILTER products BY boughtInLastMonth IS NOT NULL;
grouped = GROUP filtered BY boughtInLastMonth;
result = FOREACH grouped GENERATE
  group AS monthly_purchase,
  COUNT(filtered) AS total_products;
STORE result INTO '/amazon/output/purchase_analysis'
USING PigStorage(',');
```

Figure 5.10: Purchase Analysis Pig Script

Purchase Analysis Output



The screenshot shows a Windows Command Prompt window titled "Administrator: Command Prompt". The output of the Pig script execution is as follows:

```
2026-09-13 21:02:50,691 [main] INFO org.apache.pig.Main - Pig script completed in 10 minutes, 14 seconds and 558 milliseconds (614558 ms)

H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -ls /amazon/output/purchase_analysis
Found 2 items
-rw-r--r-- 1 user supergroup 0 2026-09-13 20:54 /amazon/output/purchase_analysis/_SUCCESS
-rw-r--r-- 1 user supergroup 20509 2026-09-13 20:54 /amazon/output/purchase_analysis/part-r-00000

H:\Amazon-Hadoop-Analysis\Pig-Scripts>hdfs dfs -cat /amazon/output/purchase_analysis/part-r-00000 | more
0,608954
1,1671
2,902
3,4575
4,28462
5,4491
6,4194
7,4001
8,5003
9,7871
10,4406
11,5519
12,5159
13,4969
14,4144
15,4136
16,4887
17,4107
18,3072
19,6701
20,1715
```

Figure 5.11: Purchase Analysis Output

6. MapReduce Analysis

Apache Hadoop MapReduce is a programming model used for processing large datasets in a distributed computing environment. It divides data processing tasks into two main phases: Mapper and Reducer.

In this project, Java-based MapReduce programs were developed to perform customized analysis on the Amazon product dataset.

The Mapper phase processes input records and generates intermediate key-value pairs. The Reducer phase receives grouped values and produces the final analytical output.

Each MapReduce program consists of three main components:

- Mapper: Extracts required information from input data.
- Reducer: Performs aggregation and generates final results.
- Driver: Configures and executes the MapReduce job.

The following MapReduce analyses were implemented:

1. Review Range Analysis
2. Top Price Analysis
3. Purchase Ranking Analysis
4. Product Popularity Analysis

6.1 Review Range Analysis

Objective

The objective of Review Range Analysis is to classify products based on the number of customer reviews. This analysis helps to understand the distribution of products according to customer engagement levels.

Methodology

The product review count was divided into three categories:

Review Range	Category
0 - 100	Low Review Products
101 - 500	Medium Review Products
500+	High Review Products

Mapper Function

The Mapper reads product records from the Amazon dataset and extracts the review count field.

The Mapper performs the following tasks:

1. Reads input CSV records.
2. Extracts product review information.
3. Assigns each product to a review range.
4. Generates key-value pairs.

Output from Mapper:

Review Range → 1

Reducer Function

The Reducer receives grouped review categories from the Mapper.

It performs counting operations to calculate the total number of products in each review category.

Output format:

Review Range	Product Count
0-100 Reviews	Count
101-500 Reviews	Count
500+ Reviews	Count

Driver Function

The Driver class performs the following operations:

The main responsibilities are:

- Creates Hadoop job configuration.
- Sets Mapper class.
- Sets Reducer class.
- Defines input path.
- Defines output path.
- Executes the MapReduce job.

Output Location

/amazon/output/review_range_analysis

Output Result

Example:

Review Range	Product Count
0-100 Reviews	607801
101-500 Reviews	34523
500+ Reviews	29956

Result Discussion

The output shows that most products belong to the lower review range category. A smaller number of products have higher review counts, indicating that only some products achieve significant customer interaction.

ReviewRangeMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class ReviewRangeMapper
extends Mapper<LongWritable, Text, Text, IntWritable> {

    public void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {

        String line = value.toString();

        if(line.startsWith("asin")) {
            return;
        }

        String[] fields = line.split(",");

        try {
            int reviews = Integer.parseInt(fields[5]);
            String range;
            if(reviews <= 100) {
                range = "0-100 Reviews";
            }
            else if(reviews <= 500) {
                range = "101-500 Reviews";
            }
            else {
                range = "500+ Reviews";
            }

            context.write(new Text(range), new IntWritable(1));
        } catch(Exception e) {
```

Figure 6.1: Review Range Mapper Code (ReviewRangeMapper.java)

ReviewRangeReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class ReviewRangeReducer
extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
    throws IOException, InterruptedException {

        int total = 0;

        for(IntWritable value : values) {
            total += value.get();
        }

        context.write(key, new IntWritable(total));
    }
}
```

Figure 6.2: Review Range Reducer Code (ReviewRangeReducer.java)

ReviewRangeDriver.java

```
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ReviewRangeDriver {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Review Range Analysis");

        job.setJarByClass(ReviewRangeDriver.class);

        job.setMapperClass(ReviewRangeMapper.class);
        job.setReducerClass(ReviewRangeReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

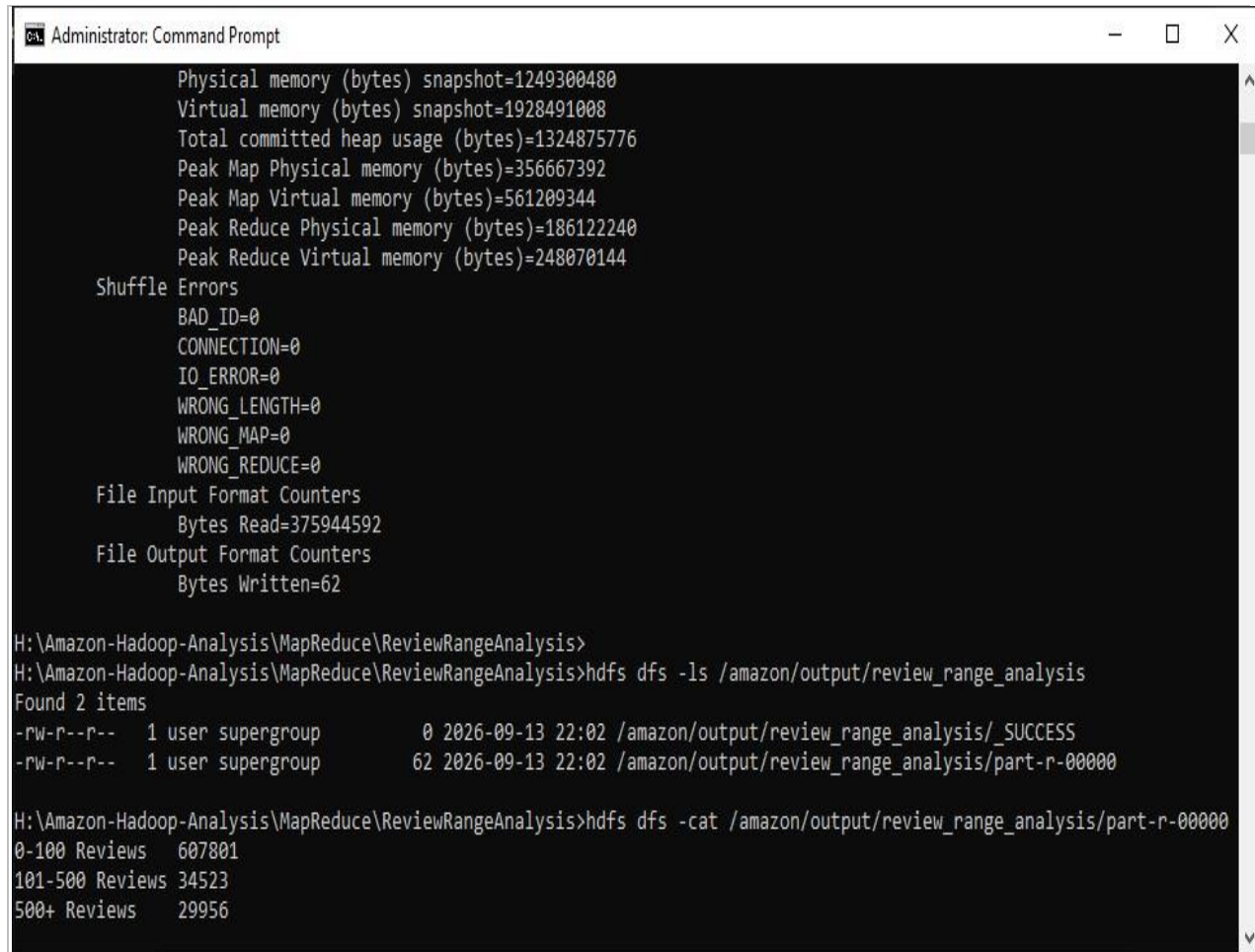
        FileInputFormat.addInputPath(job, new Path(args[0]));

        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Figure 6.3: Review Range Driver Code (ReviewRangeDriver.java)

Review Range Output



```
Physical memory (bytes) snapshot=1249300480
Virtual memory (bytes) snapshot=1928491008
Total committed heap usage (bytes)=1324875776
Peak Map Physical memory (bytes)=356667392
Peak Map Virtual memory (bytes)=561209344
Peak Reduce Physical memory (bytes)=186122240
Peak Reduce Virtual memory (bytes)=248070144

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
  Bytes Read=375944592
File Output Format Counters
  Bytes Written=62

H:\Amazon-Hadoop-Analysis\MapReduce\ReviewRangeAnalysis>
H:\Amazon-Hadoop-Analysis\MapReduce\ReviewRangeAnalysis>hdfs dfs -ls /amazon/output/review_range_analysis
Found 2 items
-rw-r--r--  1 user supergroup      0 2026-09-13 22:02 /amazon/output/review_range_analysis/_SUCCESS
-rw-r--r--  1 user supergroup    62 2026-09-13 22:02 /amazon/output/review_range_analysis/part-r-00000

H:\Amazon-Hadoop-Analysis\MapReduce\ReviewRangeAnalysis>hdfs dfs -cat /amazon/output/review_range_analysis/part-r-00000
0-100 Reviews  607801
101-500 Reviews 34523
500+ Reviews  29956
```

Figure 6.4: Review Range Analysis Output

6.2 Top Price Analysis

Objective

The objective of Top Price Analysis is to identify products with the highest price values from the Amazon product dataset. This analysis helps to understand the distribution of expensive products and identify products with higher market prices.

The analysis was performed using Hadoop MapReduce.

Methodology

The following steps were followed:

1. Read product information from the Amazon dataset.
2. Extract product ID (asin) and product price (price).
3. Convert price values into numerical format.

4. Process product prices using Mapper and Reducer.
5. Identify products with higher price values.
6. Store the final output in HDFS.

Mapper Function

The Mapper reads each product record from the input dataset.

It extracts:

- Product ID (asin)
- Product Price (price)

The Mapper generates key-value pairs:

Product_ID → Price

The Mapper performs the following operations:

1. Reads CSV input records.
2. Skips the header row.
3. Extracts product ID and price fields.
4. Removes unwanted symbols from price values.
5. Converts price into numeric format.
6. Sends the product ID and price to the Reducer.

Reducer Function

The Reducer receives product IDs with their corresponding price values.

It processes the received values and maintains the highest price products.

The Reducer uses a sorting mechanism to keep the top expensive products.

The final output format:

Product ID	Price
Product ID	Product Price

Driver Function

The Driver class manages the execution of the MapReduce job.

The main responsibilities are:

- Create Hadoop configuration.
- Initialize MapReduce job.
- Assign Mapper class.
- Assign Reducer class.
- Define input path.
- Define output path.
- Execute the job.

Output Location

/amazon/output/top_price_analysis

Output Result

Example output:

Product ID	Price
B08RVX2V6B	999.99
B07XXXXXXXX	899.99
B09XXXXXXXX	799.99

Result Discussion

The Top Price Analysis successfully identifies products with higher price values from the Amazon dataset.

The output provides information about expensive products and demonstrates how MapReduce can process large product datasets to extract specific information based on price attributes.

TopPriceMapper.java

```
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class TopPriceMapper extends Mapper<LongWritable,Text,Text,DoubleWritable>{

    public void map(LongWritable key,Text value,Context context)
    throws IOException,InterruptedException{

        String line=value.toString();

        if(line.startsWith("asin"))
            return;

        String[] fields=line.split(",");

        try{
            String productId=fields[0];
            String priceText=fields[6];

            priceText=priceText.replace("$","")
                .replace(",","")
                .trim();

            double price=Double.parseDouble(priceText);

            context.write(new Text(productId),
                new DoubleWritable(price));

        }catch(Exception e)
        {
        }

    }
}
```

Figure 6.5: Top Price Mapper Code (TopPriceMapper.java)

TopPriceReducer.java

```
import java.io.IOException;
import java.util.TreeMap;
import java.util.Map;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class TopPriceReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    private TreeMap<Double, Text> topPrices = new TreeMap<>();
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context)
    throws IOException, InterruptedException {

        for(DoubleWritable value : values) {

            double price = value.get();

            topPrices.put(price, new Text(key));

            if(topPrices.size() > 20) {
                topPrices.remove(topPrices.firstKey());
            }
        }
    }
    protected void cleanup(Context context)
    throws IOException, InterruptedException {

        for(Map.Entry<Double, Text> entry : topPrices.descendingMap().entrySet()) {

            context.write(entry.getValue(),
                new DoubleWritable(entry.getKey()));

        }
    }
}
```

Figure 6.6: Top Price Reducer Code (TopPriceReducer.java)

TopPriceDriver.java

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class TopPriceDriver{

    public static void main(String[] args)throws Exception{

        Configuration conf=new Configuration();

        Job job=Job.getInstance(conf,"Top Expensive Products");

        job.setJarByClass(TopPriceDriver.class);

        job.setMapperClass(TopPriceMapper.class);
        job.setReducerClass(TopPriceReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(DoubleWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job,new Path(args[0]));

        FileOutputFormat.setOutputPath(job,new Path(args[1]));

        System.exit(job.waitForCompletion(true)?0:1);

    }
}
```

Figure 6.7: Top Price Driver Code (TopPriceDriver.java)

Top Price Analysis Output

```
H:\Amazon-Hadoop-Analysis\MapReduce\TopPriceAnalysis>hdfs dfs -ls /amazon/output/top_price_analysis
Found 2 items
-rw-r--r--  1 user supergroup      0 2026-09-13 22:24 /amazon/output/top_price_analysis/_SUCCESS
-rw-r--r--  1 user supergroup 479 2026-09-13 22:24 /amazon/output/top_price_analysis/part-r-00000

H:\Amazon-Hadoop-Analysis\MapReduce\TopPriceAnalysis>hdfs dfs -cat /amazon/output/top_price_analysis/part-r-00000
B08RXV2Z6B      Infinity
B087M62BBV      3.6291114101E10
B08VVDCJTJ      1.0000013001E10
B087LRPRJV      9.114638812E9
B08RKSJN8Y      8.19100213E9
B07YS6W2FN      7.241509402E9
B07VG9YBCQ      5.303918847E9
B083DQVT8L      5.303283287E9
B008LT7668      2.08272E9
B0C9931CLZ      1.648201174E9
B0C9PQJGKC      4.40003993E8
B07XTNMCN6      3.16075104E8
B087LT37M9      3.16075102E8
B01JMAZEA       3.01205E8
B07RH9ML71      2.42294501E8
B01CC7QFKC      2.42047801E8
B0CGWRMML8      2.42017801E8
B0BZ777WLW      2.420178E8
B07CSG3GVG      2.41636701E8
B09YY6RBW2      2.40323007E8
```

Figure 6.8: Top Price Analysis Output

6.3 Purchase Ranking Analysis

Objective

The objective of Purchase Ranking Analysis is to identify products based on their recent purchasing activity. This analysis uses the `boughtInLastMonth` attribute from the Amazon product dataset to understand which products have higher customer purchase activity.

The analysis was performed using Hadoop MapReduce.

Methodology

The following steps were followed:

1. Read product records from the Amazon dataset.
2. Extract product ID (`asin`) and monthly purchase count (`boughtInLastMonth`).
3. Generate key-value pairs using the Mapper.
4. Aggregate purchase values using the Reducer.
5. Generate the final purchase ranking output.
6. Store the result in HDFS.

Mapper Function

The Mapper reads each product record from the input dataset.

It extracts:

- Product ID (`asin`)
- Number of purchases in the last month (`boughtInLastMonth`)

The Mapper generates key-value pairs:

Product_ID → Monthly Purchase Count

The Mapper performs the following operations:

1. Reads CSV product records.
2. Skips invalid records.
3. Extracts product ID and purchase information.
4. Converts purchase values into numerical format.
5. Sends data to the Reducer for aggregation.

Reducer Function

The Reducer receives product IDs with their corresponding purchase values.

It performs aggregation by adding purchase counts belonging to the same product.

The final output format:

Product ID	Total Purchase
Product ID	Purchase Count

Driver Function

The Driver class controls the MapReduce job execution.

The main responsibilities are:

- Create Hadoop job configuration.
- Set Mapper class.
- Set Reducer class.
- Define input path.
- Define output path.
- Execute the MapReduce program.

Output Location

/amazon/output/purchase_ranking_analysis

Output Result

Example output:

Product ID	Total Purchase
B001XXXXX	5000
B002XXXXX	4200
B003XXXXX	3000

Result Discussion

The Purchase Ranking Analysis identifies products with higher recent purchase activity.

The output helps to understand customer buying patterns and provides information about products that receive more attention from customers during the last month.

This analysis demonstrates how MapReduce can process large product datasets and generate useful purchase-related insights.

PurchaseRankingMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class PurchaseMapper extends Mapper<LongWritable,Text,Text,IntWritable>{

    public void map(LongWritable key,Text value,Context context)
    throws IOException,InterruptedException{

        String line=value.toString();

        if(line.startsWith("asin"))
            return;

        String[] fields=line.split(",");

        try{

            String product=fields[0];
            int purchase=Integer.parseInt(fields[10]);

            context.write(new Text(product),
            new IntWritable(purchase));

        }catch(Exception e){
        }

    }
}
```

Figure 6.9: Purchase Ranking Mapper Code (PurchaseRankingMapper.java)

PurchaseRankingReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class PurchaseReducer extends Reducer<Text,IntWritable,Text,IntWritable>{

    public void reduce(Text key,Iterable<IntWritable> values,Context context)
    throws IOException,InterruptedException{

        int total=0;

        for(IntWritable value:values){
            total+=value.get();
        }
        context.write(key,new IntWritable(total));

    }
}
```

Figure 6.10: Purchase Ranking Reducer Code (PurchaseRankingReducer.java)

PurchaseRankingDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class PurchaseDriver{

    public static void main(String[] args)throws Exception{

        Configuration conf=new Configuration();

        Job job=Job.getInstance(conf,"Purchase Ranking Analysis");

        job.setJarByClass(PurchaseDriver.class);

        job.setMapperClass(PurchaseMapper.class);
        job.setReducerClass(PurchaseReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job,new Path(args[0]));

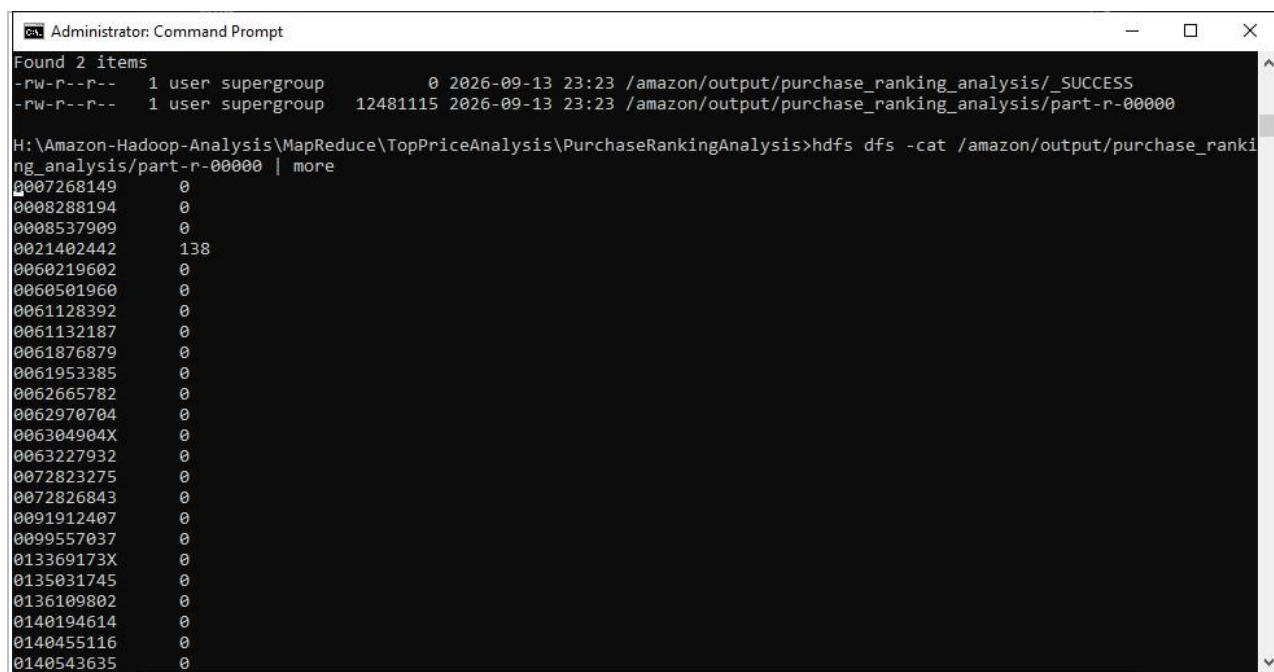
        FileOutputFormat.setOutputPath(job,new Path(args[1]));

        System.exit(job.waitForCompletion(true)?0:1);

    }
}
```

Figure 6.11: Purchase Ranking Driver Code (*PurchaseRankingDriver.java*)

Purchase Ranking Analysis Output



```
Administrator: Command Prompt
Found 2 items
-rw-r--r-- 1 user supengroup 0 2026-09-13 23:23 /amazon/output/purchase_ranking_analysis/_SUCCESS
-rw-r--r-- 1 user supengroup 12481115 2026-09-13 23:23 /amazon/output/purchase_ranking_analysis/part-r-00000

H:\Amazon-Hadoop-Analysis\MapReduce\TopPriceAnalysis\PurchaseRankingAnalysis>hdfs dfs -cat /amazon/output/purchase_ranki
ng_analysis/part-r-00000 | more
0007268149 0
0008288194 0
0008537909 0
0021402442 138
0060219602 0
0060501960 0
0061128392 0
0061132187 0
0061876879 0
0061953385 0
0062665782 0
0062970704 0
006304904X 0
0063227932 0
0072823275 0
0072826843 0
0091912407 0
0099557037 0
013369173X 0
0135031745 0
0136109802 0
0140194614 0
0140455116 0
0140543635 0
```

Figure 6.12: Purchase Ranking Analysis Output

6.4 Product Popularity Analysis

Objective

The objective of Product Popularity Analysis is to identify popular products based on the number of customer reviews. Products with higher review counts indicate greater customer interaction and engagement. This analysis helps to understand which products receive more attention from customers within the Amazon dataset.

The analysis was performed using Hadoop MapReduce.

Methodology

The following steps were followed:

1. Read product information from the Amazon product dataset.
2. Extract product ID (asin) and review count (reviews).
3. Generate key-value pairs using the Mapper.
4. Aggregate review values using the Reducer.
5. Generate the final product popularity output.
6. Store the result in HDFS.

Mapper Function

The Mapper reads each product record from the input dataset.

It extracts:

- Product ID (asin)
- Customer review count (reviews)

The Mapper generates key-value pairs:

Product_ID → Review_Count

The Mapper performs the following operations:

1. Reads CSV input records.
2. Skips the header row.
3. Extracts product ID and review count.
4. Converts review values into numerical format.
5. Sends product review information to the Reducer.

Reducer Function

The Reducer receives product IDs with their corresponding review values.

It performs aggregation by calculating the total review count associated with each product.

The final output format:

Product ID	Review Count
Product ID	Total Reviews

Driver Function

The Driver class manages the execution of the MapReduce job.

The main responsibilities are:

- Create Hadoop job configuration.
- Set Mapper class.
- Set Reducer class.
- Define input location.
- Define output location.
- Execute the MapReduce process.

Output Location

/amazon/output/product_popularity_analysis

Output Result

Example output:

Product ID	Review Count
0143130722	53598
B001XXXXX	45000
B002XXXXX	38000

Result Discussion

The Product Popularity Analysis successfully identifies products with higher customer review counts.

The output demonstrates customer engagement patterns within the Amazon dataset. Products with more reviews represent higher interaction from customers compared with products having fewer reviews.

This analysis shows how Hadoop MapReduce can process large-scale product information and extract useful popularity-related insights.

PopularityMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class PopularityMapper extends Mapper<LongWritable,Text,Text,IntWritable>{

    public void map(LongWritable key,Text value,Context context)
    throws IOException,InterruptedException{

        String line=value.toString();

        if(line.startsWith("asin"))
            return;

        String[] fields=line.split(",");

        try{

            String product=fields[0];
            int reviews=Integer.parseInt(fields[5]);

            context.write(new Text(product),
            new IntWritable(reviews));

        }catch(Exception e){
        }

    }
}
```

Figure 6.13: Product Popularity Mapper Code (PopularityMapper.java)

PopularityReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class PopularityReducer extends Reducer<Text,IntWritable,Text,IntWritable>{

    public void reduce(Text key,Iterable<IntWritable> values,Context context)
    throws IOException,InterruptedException{

        int total=0;

        for(IntWritable value:values){
            total+=value.get();
        }

        context.write(key,new IntWritable(total));

    }
}
```

Figure 6.14: Product Popularity Reducer Code (PopularityReducer.java)

PopularityDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class PopularityDriver{

    public static void main(String[] args)throws Exception{

        Configuration conf=new Configuration();

        Job job=Job.getInstance(conf,"Product Popularity Analysis");

        job.setJarByClass(PopularityDriver.class);

        job.setMapperClass(PopularityMapper.class);
        job.setReducerClass(PopularityReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job,new Path(args[0]));

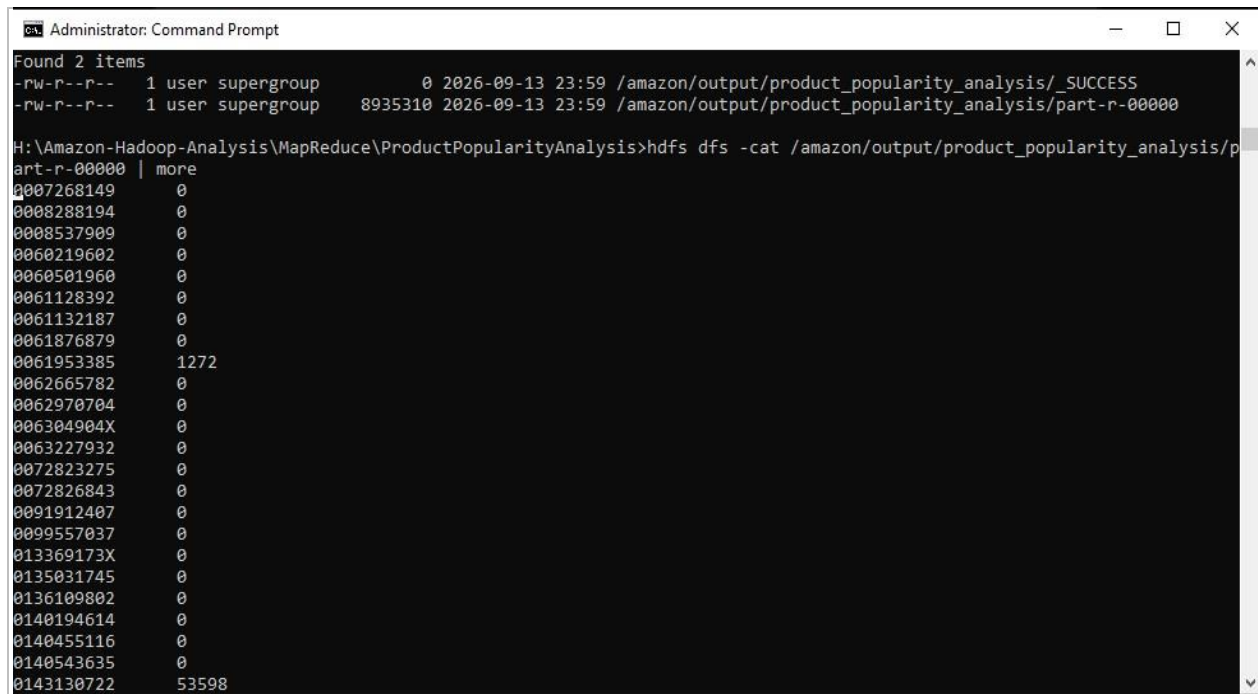
        FileOutputFormat.setOutputPath(job,new Path(args[1]));

        System.exit(job.waitForCompletion(true)?0:1);

    }
}
```

Figure 6.15: Product Popularity Driver Code (PopularityDriver.java)

Product Popularity Analysis Output



```
Administrator: Command Prompt

Found 2 items
-rw-r--r-- 1 user supergroup 0 2026-09-13 23:59 /amazon/output/product_popularity_analysis/_SUCCESS
-rw-r--r-- 1 user supergroup 8935310 2026-09-13 23:59 /amazon/output/product_popularity_analysis/part-r-00000

H:\Amazon-Hadoop-Analysis\MapReduce\ProductPopularityAnalysis>hdfs dfs -cat /amazon/output/product_popularity_analysis/p
art-r-00000 | more
0007268149 0
0008288194 0
0008537909 0
0060219602 0
0060501960 0
0061128392 0
0061132187 0
0061876879 0
0061953385 1272
0062665782 0
0062970704 0
006304904X 0
0063227932 0
0072823275 0
0072826843 0
0091912407 0
0099557037 0
013369173X 0
0135031745 0
0136109802 0
0140194614 0
0140455116 0
0140543635 0
0143130722 53598
```

Figure 6.16: Product Popularity Analysis Output

7. Results and Discussion

The project successfully analyzed a large Amazon product dataset using Hadoop ecosystem technologies. Apache Pig and MapReduce were applied to process and analyze different product attributes.

Apache Pig was used for performing high-level data analysis operations. The Pig-based analyses provided information about product ratings, customer reviews, product categories, pricing patterns, and purchase activities.

The following results were obtained from Apache Pig analysis:

Analysis	Purpose	Result
Rating Analysis	Analyze product distribution based on star ratings	Identified product counts for different rating values
Review Analysis	Analyze customer review information	Extracted product-wise review details
Category Analysis	Analyze product distribution across categories	Identified number of products in different categories
Price Analysis	Analyze product price distribution	Identified product distribution based on price values
Purchase Analysis	Analyze recent purchase activity	Identified product counts based on monthly purchase values

MapReduce programs were developed to perform customized analysis tasks.

The following results were obtained from MapReduce analysis:

Analysis	Purpose	Result
Review Range Analysis	Classify products according to review counts	Products were divided into low, medium, and high review categories
Top Price Analysis	Identify expensive products	Generated products with higher price values
Purchase Ranking Analysis	Analyze purchasing activity	Identified products based on monthly purchase counts
Product Popularity Analysis	Identify popular products	Generated product popularity information using review counts

The Review Range Analysis showed that most products belong to the lower review category, while fewer products have very high review counts. This indicates that only a limited number of products receive significant customer interaction.

The Top Price Analysis successfully identified products with higher price values from the dataset. This demonstrates the ability of MapReduce to process large datasets and extract specific information based on user requirements.

The Purchase Ranking Analysis provided information about products with higher recent purchase activity. This helps to understand customer purchasing behavior.

The Product Popularity Analysis identified products with higher numbers of customer reviews. Products with more reviews represent greater customer engagement compared with products having fewer reviews.

Overall, the project demonstrates that Hadoop can efficiently process large-scale product datasets and generate meaningful analytical results using Apache Pig and MapReduce.

8. Conclusion

This project demonstrated the practical application of Hadoop ecosystem technologies for analyzing large-scale Amazon product data. The project successfully showed how big data tools can be used to store, process, and analyze a large volume of product information in an efficient and distributed computing environment.

Apache Hadoop provided the foundation for handling large datasets through Hadoop Distributed File System (HDFS) and distributed processing capabilities. The Amazon product dataset containing approximately 1.4 million records was stored and processed using Hadoop-based technologies. This allowed efficient management and analysis of large-scale product information.

Apache Pig was used to perform high-level data analysis operations through Pig Latin scripts. It simplified the process of extracting useful information from raw product data. Using Apache Pig, different analyses were performed, including rating analysis, review analysis, category analysis, price analysis, and purchase analysis. These analyses provided valuable information about product distribution, customer engagement, pricing patterns, and purchasing activities.

Java-based MapReduce programs were developed to perform customized analytical operations on the dataset. The implemented MapReduce analyses included review range classification, top price identification, purchase ranking analysis, and product popularity analysis. These programs demonstrated how customized data processing tasks can be executed efficiently using the MapReduce programming model.

The generated outputs provided useful information about product characteristics and customer behavior. The analysis helped identify patterns related to product ratings, customer reviews, product prices, purchase activities, and popularity levels. These results demonstrate the capability of Hadoop technologies to transform large raw datasets into meaningful analytical information.

Through this project, practical knowledge of big data storage, distributed processing, Apache Pig scripting, and Java MapReduce programming was developed. The project also demonstrated the importance of Hadoop-based solutions for handling large datasets generated by modern digital platforms.

In conclusion, Hadoop-based data analysis provides an effective approach for processing and analyzing large-scale datasets. The combination of HDFS, Apache Pig, and MapReduce enables efficient data management and supports data-driven decision-making by extracting meaningful insights from complex datasets.

References:

- 1. Java Documentation** <https://docs.oracle.com/en/java/>
- 2. Apache Hadoop Documentation** <https://hadoop.apache.org/docs/>
- 3. Apache Pig Documentation** <https://pig.apache.org/docs/>
- 4. Dataset Description** <https://www.kaggle.com/datasets/aaronfriasr/amazon-products-dataset>
- 5. Github link** <https://github.com/Sushen1157/Amazon-Product-Data-Analysis-Using-Hadoop/tree/main/MapReduce/ReviewRangeAnalysis>